CSC 4814 & 6814 Efficient AI

Lecture 06

Dr. Haoxin Wang

Assistant Professor, GSU

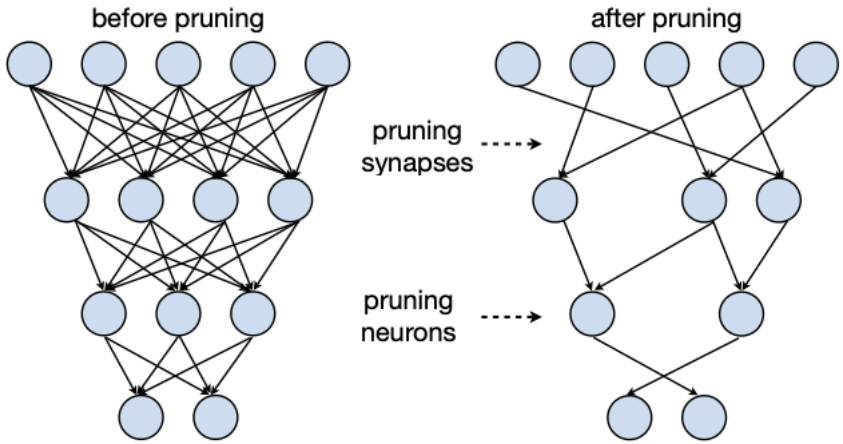
https://www.amai-gsu.us/

haoxinwang@gsu.edu

This course draws inspiration and selected materials from Prof. Song Han's MIT course on TinyML and Efficient Deep Learning Computing. Credit is gratefully acknowledged.

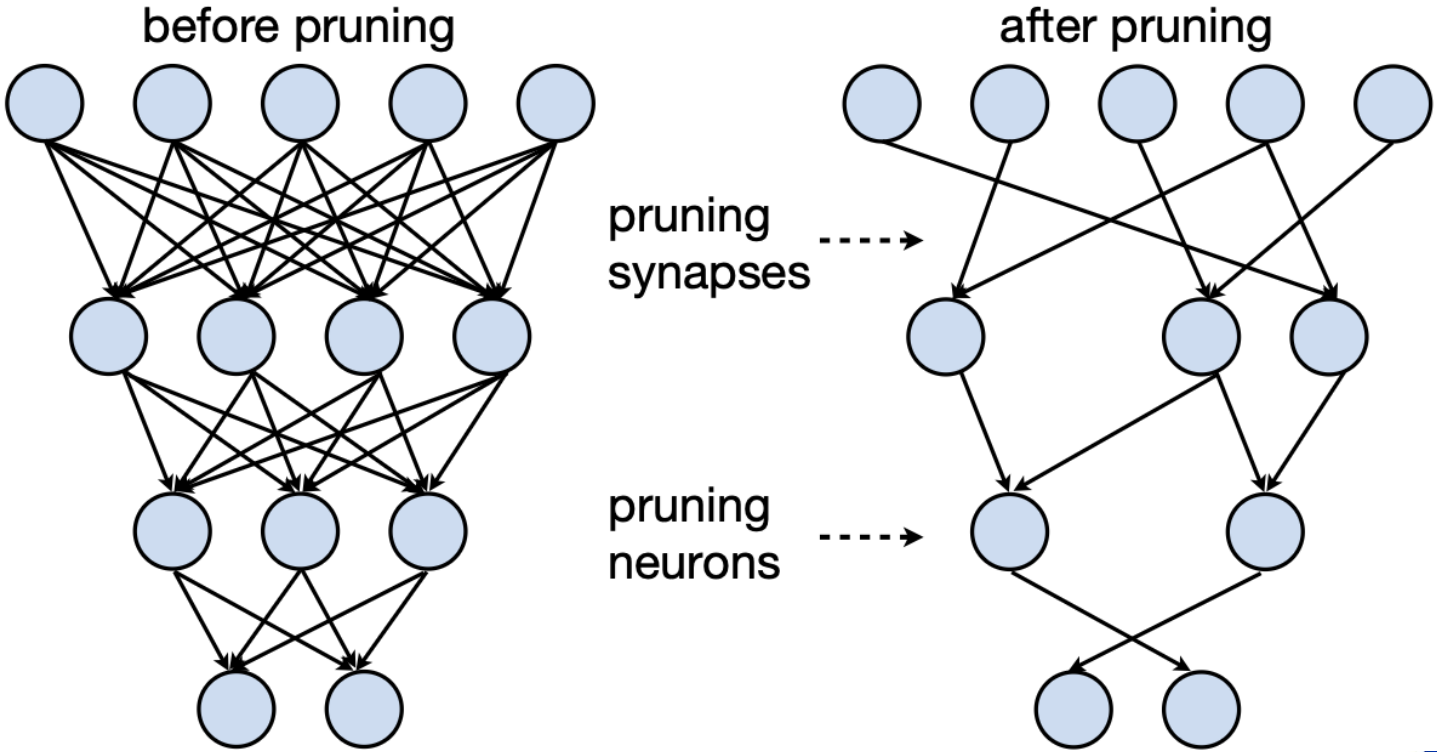
Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
  - How should we improve performance of pruned models?



Neural Network Pruning

Make neural network smaller by removing parameters (synapses) and activations (neurons)



Neural Network Pruning

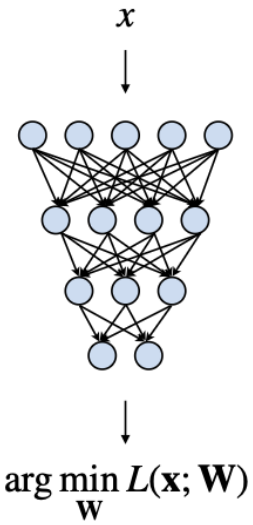
- In general, we could formulate the pruning as follows:

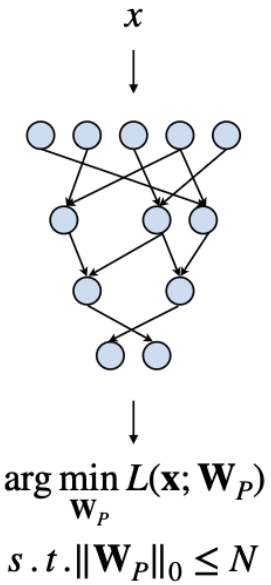
$$\arg\min_{\mathbf{W}_P} L(\mathbf{x}; \mathbf{W}_P)$$

subject to

$$\|\mathbf{W}_p\|_0 < N$$

  - L represents the objective function for neural network training;
  - $\mathbf{x}$ is input, $\mathbf{W}$ is original weights, $\mathbf{W}_P$ is pruned weights;
  - $\|\mathbf{W}_p\|_0$ calculates the #nonzeros in W_P, and N is the target #nonzeros.





Pruning Granularity

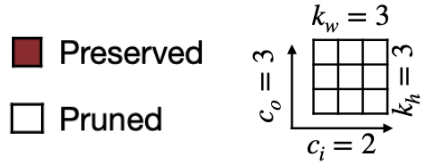
Pruning can be performed at different granularities, from structured to non-structured.



Pruning at Different Granularities

The case of convolutional layers

- Some of the commonly used pruning granularities

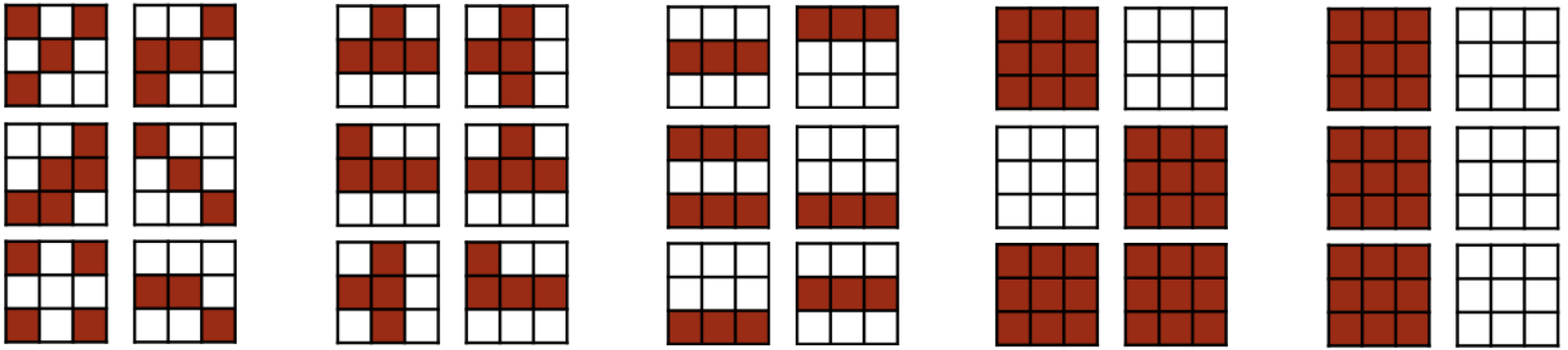


Pros?

Cons?

Irregular

Regular



Fine-grained Pruning

Pattern-based Pruning

Vector-level Pruning

Kernel-level Pruning

Channel-level Pruning

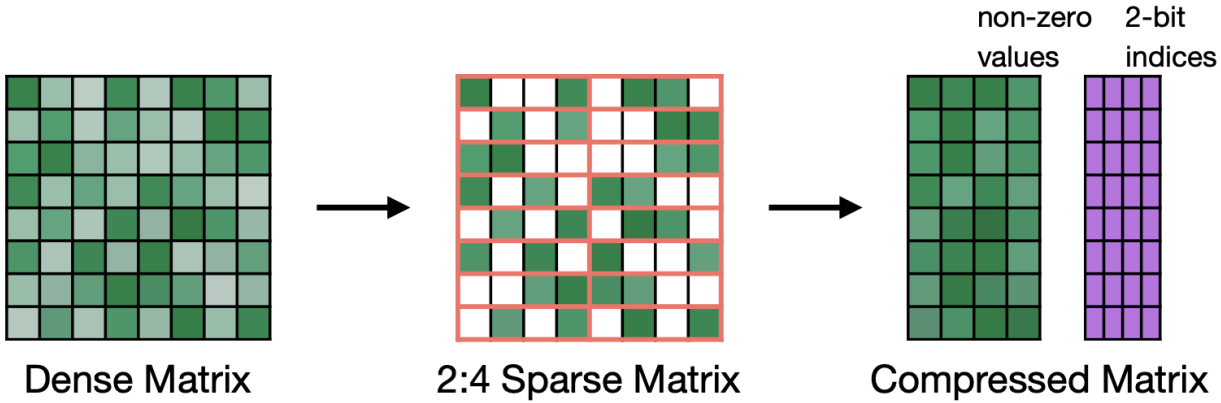
like Tetris :)

Exploring the granularity of sparsity in convolutional neural networks [Mao *et al.*, CVPR-W]

Pruning at Different Granularities

Let's look into some cases

- **Pattern-based Pruning: N:M sparsity**
  - N:M sparsity means that in each contiguous M elements, N of them is pruned
  - A classic case is 2:4 sparsity (50% sparsity)
  - It is supported by NVIDIA's Ampere GPU Architecture, which delivers up to 2x speed up

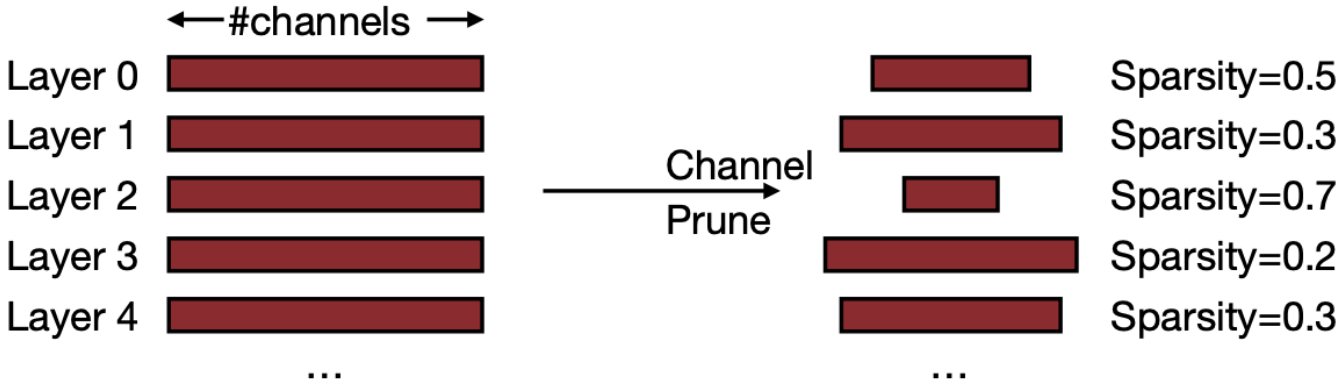


Accelerating Inference with Sparsity Using the NVIDIA Ampere Architecture and NVIDIA TensorRT

Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**
  - Pro: Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
  - Con: smaller compression ratio

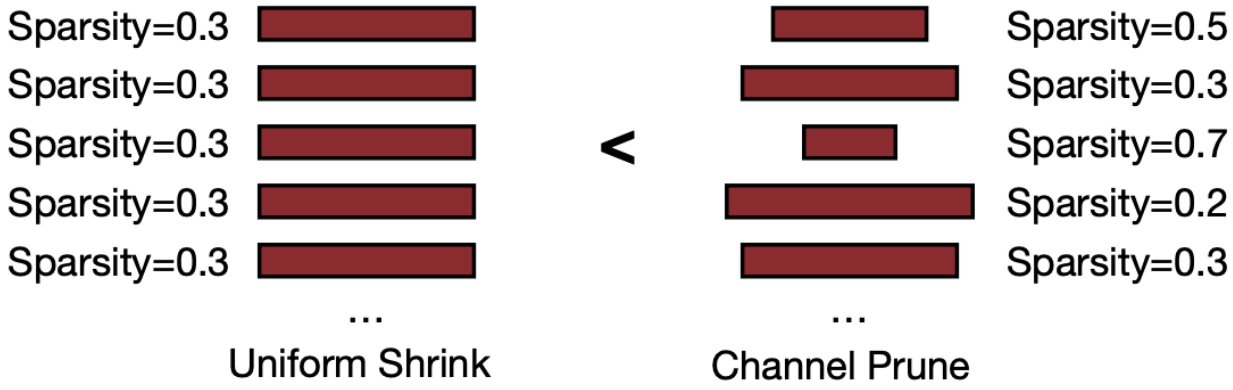


Pruning at Different Granularities

Let's look into some cases

- **Channel Pruning**
  - Pro: Direct speed up due to reduced channel numbers (leading to an NN with smaller #channels)
  - Con: smaller compression ratio

We will later discuss how to find sparsity ratios



Pruning Criterion

What synapses and neurons should we prune?



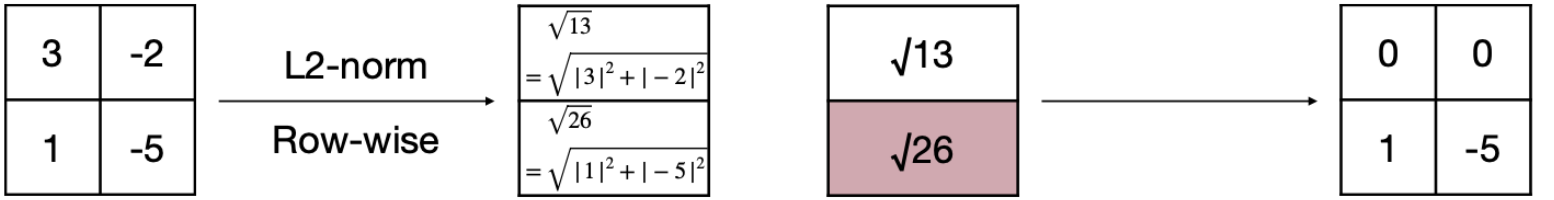
Magnitude-based Pruning

A heuristic pruning criterion

- Magnitude-based pruning considers weights with ***larger absolute values*** are more important than other weights.
- Magnitude is also known as L_p-norm defined as,

$$\|\mathbf{W}^{(S)}\|_p = \left(\sum_{i \in S} |w_i|^p \right)^{\frac{1}{p}}, \text{ where } \mathbf{W}^{(S)} \text{ is a structural set of parameters}$$

- **Example**

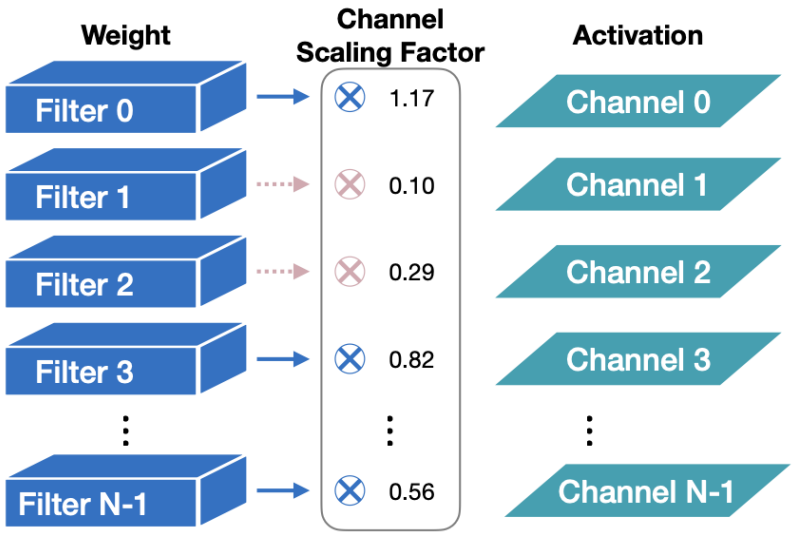


Learning Structured Sparsity in Deep Neural Networks [Wen *et al.*, NeurIPS 2016]

Scaling-based Pruning

Pruning criterion for filter pruning

- A scaling factor is associated with each filter (*i.e.*, output channel) in convolutional layers
  - The scaling factor is multiplied to the output of that channel
  - The scaling factors are trainable parameters



Learning Efficient Convolutional Networks through Network Slimming [Liu *et al.*, ICCV 2017]

Second-Order-based Pruning

Minimize the error on loss function introduced by pruning synapses

- The induced error can be approximated by a Taylor series.

$$\delta L = L(\mathbf{x}; \mathbf{W}) - L(\mathbf{x}; \mathbf{W}_P = \mathbf{W} - \delta\mathbf{W}) = \sum_i g_i w_i + \frac{1}{2}\sum_i h_{ii}\delta w_i^2 + \frac{1}{2}\sum_{i \neq j} h_{ij}\delta w_i \delta w_j + O(\|\mathbf{W}\|^3)$$

where

$$g_i = \frac{\partial L}{\partial w_i}, \; h_{i,j} = \frac{\partial^2 L}{\partial w_i \partial w_j}$$

- Optimal Brain Damage assumes that
  - The objective function L is nearly quadratic: the last term is neglected
  - The neural network training has converged: first-order terms are neglected
  - The error caused by deleting each parameter is independent: cross terms are neglected

$$\delta L_i = L(\mathbf{x}; \mathbf{W}) - L(\mathbf{x}; \mathbf{W}_P \,|\, w_i = 0) \approx \frac{1}{2} h_{ii} w_i^2$$

the estimated increase in loss if weight w_i is pruned (set to zero).

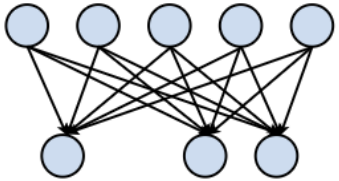
Optimal Brain Damage [LeCun *et al.*, NeurIPS 1989]

Selection of Neurons (Activations) to Prune

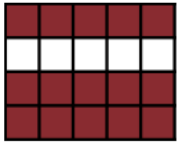
- When removing neurons from a neural network model,
  - ***the less useful*** the neurons being removed are,
  - the better the performance of pruned neural network is.

Neuron pruning is coarse-grained weight pruning

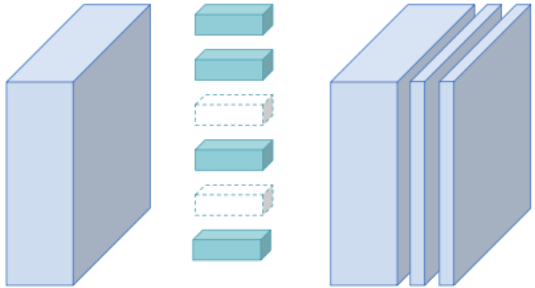
Neuron Pruning in Linear Layer

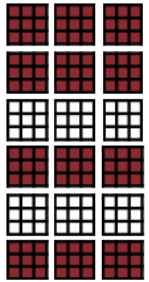


Weight Matrix



Channel Pruning in Convolution Layer





Percentage-of-Zero-Based Pruning

- ReLU activation will generate zeros in the output activation.
- Similar to magnitude of weights, the Average Percentage of Zero activations (APoZ) can be exploited to measure the importance of the neurons.
- The smaller APoZ is, the more importance the neuron has.

Output Activations

Batch = 2

Width = 4, Height = 4, Channel = 3

0	0.1	0.5	1
1.2	0.6	0.3	0.2
0	0.5	**0**	0.3
0.2	**0**	**0**	0.8

0.1	0.5	**0**	**0**
0.2	0.3	**0**	1
0.1	**0**	**0**	0.5
0.1	0.6	0.7	0.1

0	**0**	0.8	**0**
0.7	**0**	0.6	0.1
1.2	1	**0**	0.2
0.5	**0**	0.3	0.5

Width = 4, Height = 4, Channel = 3

0.5	**0**	0.2	0.1
0	0.2	1.2	**0**
1.2	**0**	0.2	0.3
0.2	0.4	**0**	**0**

0.1	0.5	**0**	**0**
0	0.8	**0**	1
0.1	**0**	0.1	1.0
0.2	**0**	1.0	**0**

0	0.8	0.1	**0**
0.2	**0**	**0**	0.3
0	0.4	**0**	0.5
0.2	**0**	0.3	**0**

Average Percentage of Zeros (APoZ)

$= \frac{5+6}{2 \cdot 4 \cdot 4} = \frac{11}{32}$ Channel 0

$= \frac{5+7}{2 \cdot 4 \cdot 4} = \frac{12}{32}$ Channel 1

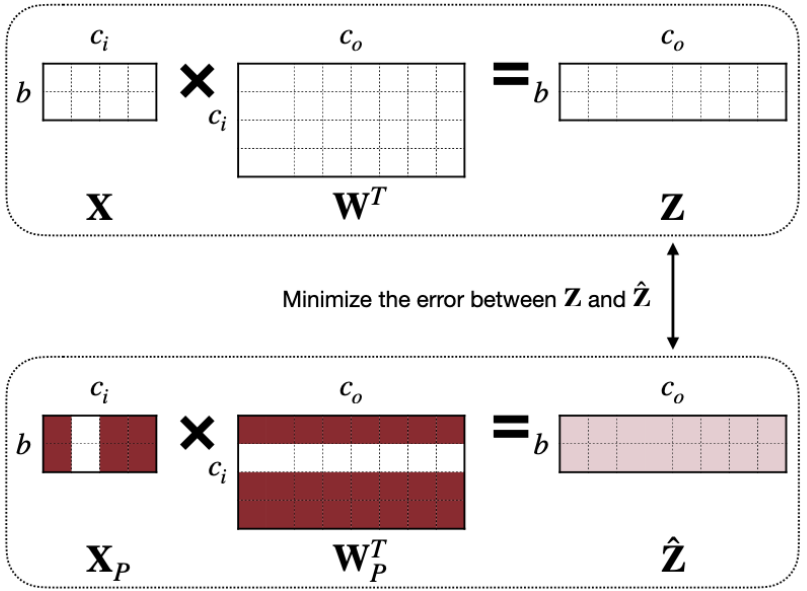
$= \frac{6+8}{2 \cdot 4 \cdot 4} = \frac{14}{32}$ Channel 2

Network Trimming: A Data-Driven Neuron Pruning Approach towards Efficient Deep Architectures [Hu *et al.*, ArXiv 2017]

Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

- Instead of considering the pruning error of the objective function $L(\mathbf{x}; \mathbf{W})$, regression-based pruning minimizes the reconstruction error of the corresponding layer's outputs.



Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

- Let

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T = \sum_{c=0}^{c_i-1} \mathbf{X}_c \mathbf{W}_c^T$$

- The problem can be formulate as

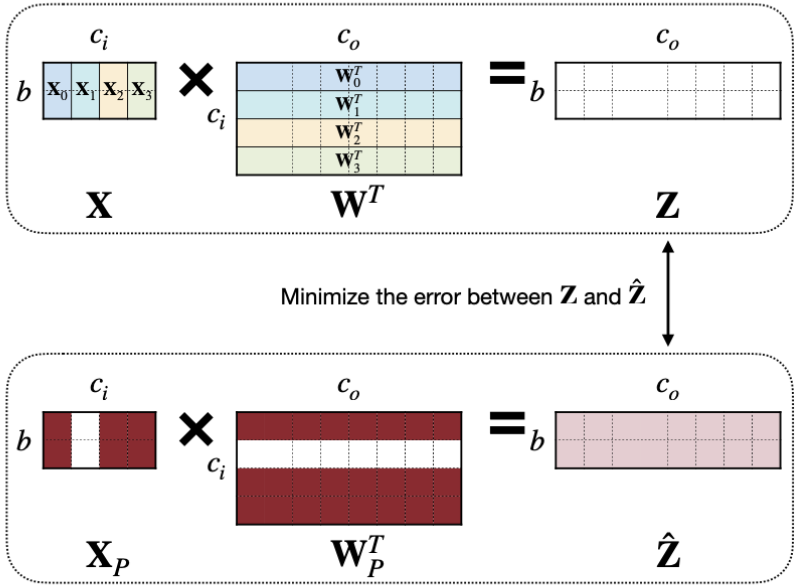
$$\arg\min_{\mathbf{W},\beta} \|\mathbf{Z} - \hat{\mathbf{Z}}\|_F^2 = \|\mathbf{Z} - \sum_{c=0}^{c_i-1} \beta_c \mathbf{X}_c \mathbf{W}_c^T\|_F^2$$

scaling factor

subject to

$$\|\beta\|_0 \le N_c$$

- β is coefficient vector of length c_i for channel selection. $\beta_c = 0$ means channel c is pruned.
- N_c is the number of nonzero channels.



Channel Pruning for Accelerating Very Deep Neural Networks [He *et al*., ICCV 2017]

Regression-based Pruning

Minimize reconstruction error of the corresponding layer's outputs

- Let

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T = \sum_{c=0}^{c_i-1} \mathbf{X}_c \mathbf{W}_c^T$$

- The problem can be formulate as

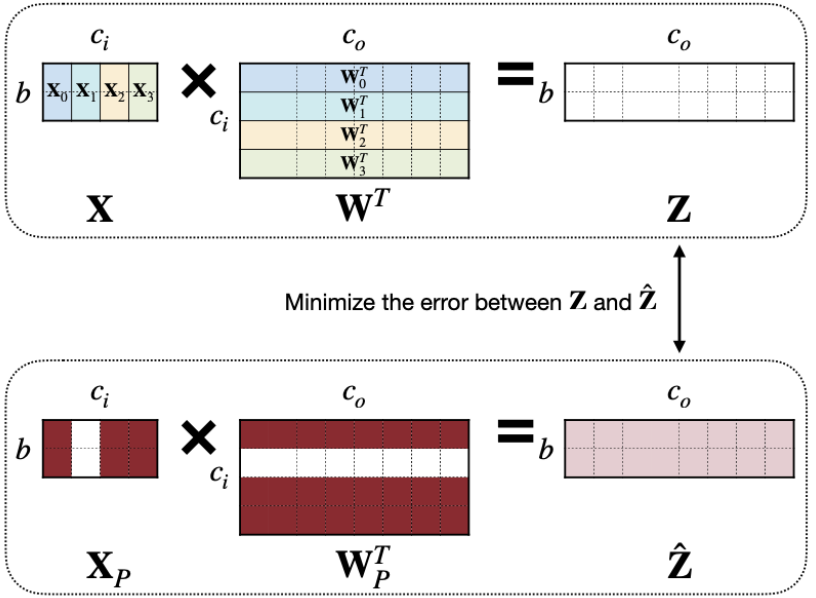
$$\arg\min_{\mathbf{W},\,\beta} \|\mathbf{Z} - \hat{\mathbf{Z}}\|_F^2 = \|\mathbf{Z} - \sum_{c=0}^{c_i-1} \beta_c \mathbf{X}_c \mathbf{W}_c^T\|_F^2$$

  subject to

$$\|\beta\|_0 \le N_c$$

- β is coefficient vector of length c_i for channel selection. $\beta_c = 0$ means channel c is pruned.
- N_c is the number of nonzero channels.

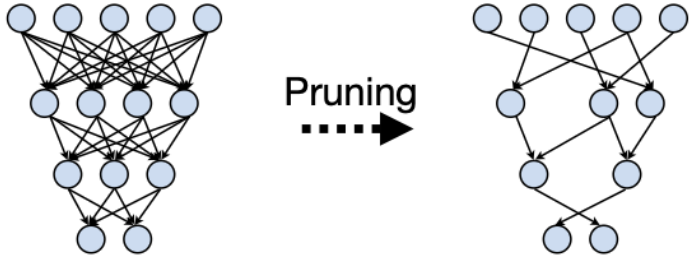
- Solve the problem by:
  - Fix $\mathbf{W}$, solve β for channel selection
  - Fix β, solve $\mathbf{W}$ to minimize reconstruction error



Channel Pruning for Accelerating Very Deep Neural Networks [He *et al.*, ICCV 2017]

Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
  - How should we improve performance of pruned models?



prune 30%?
prune 50%?
prune 70%?

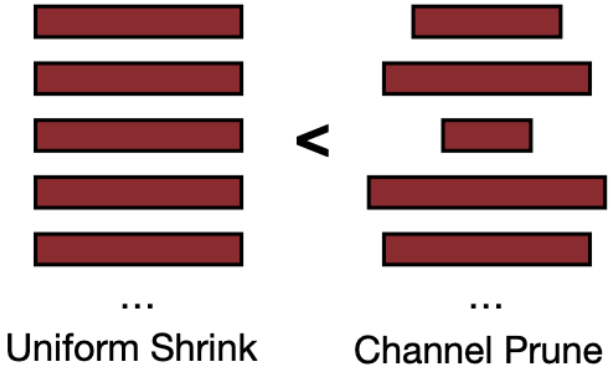
Pruning Ratio

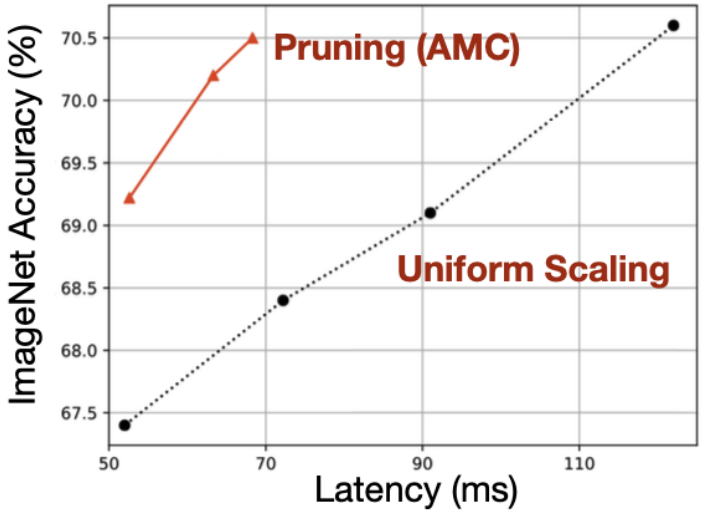
How should we find per-layer pruning ratios?



Recap

Non-uniform pruning is better than uniform shrinking





Question: how should we find ratios for each layer?

AMC: Automl for Model Compression and Acceleration on Mobile Devices [He *et al.*, ECCV 2018]

Finding Pruning Ratios

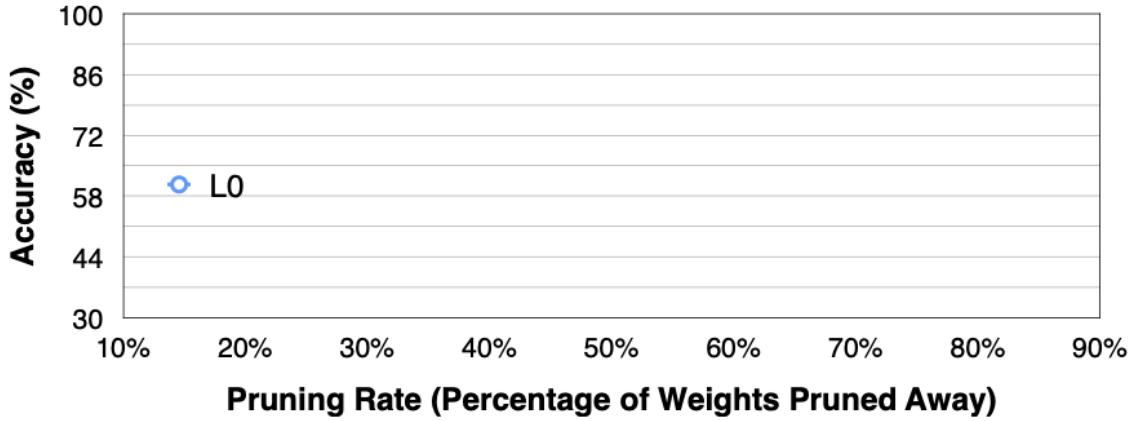
Analyze the sensitivity of each layer

- We need different pruning ratios for each layer since different layers have different **sensitivity**
  - Some layers are more sensitive (e.g., first layer)
  - Some layers are more redundant
- We can perform **sensitivity analysis** to determine the per-layer pruning ratio

Finding Pruning Ratios

Analyze the sensitivity of each layer

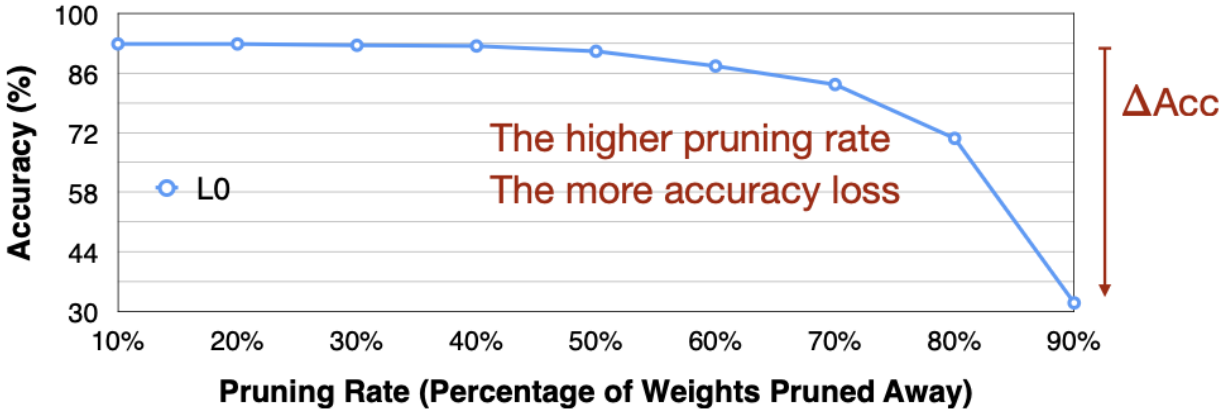
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model



Finding Pruning Ratios

Analyze the sensitivity of each layer

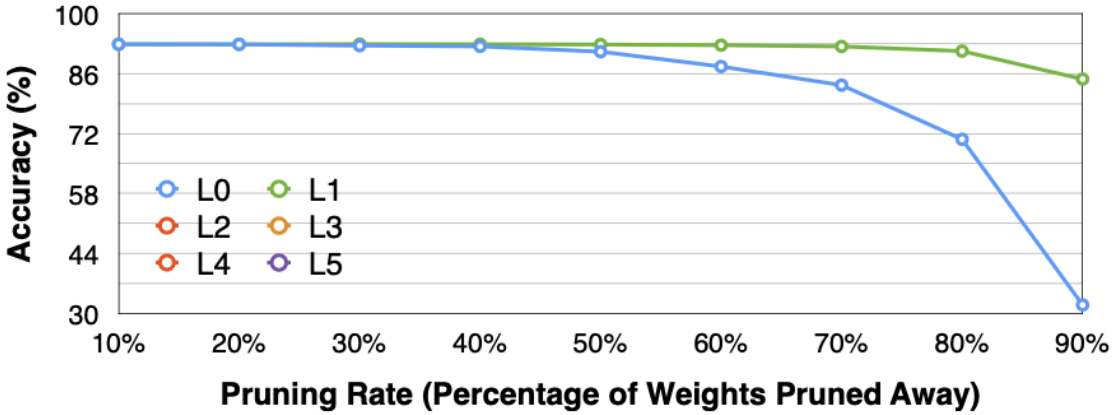
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio



Finding Pruning Ratios

Analyze the sensitivity of each layer

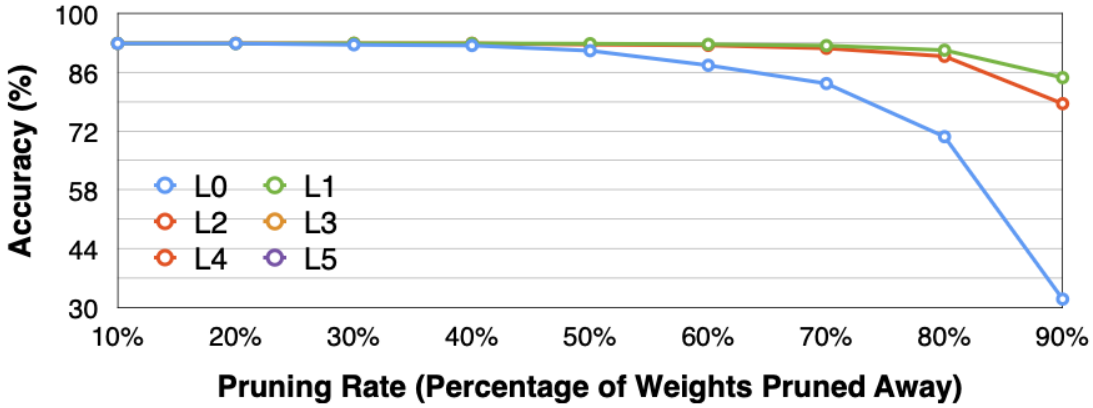
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0,0.1,0.2,...,0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers



Finding Pruning Ratios

Analyze the sensitivity of each layer

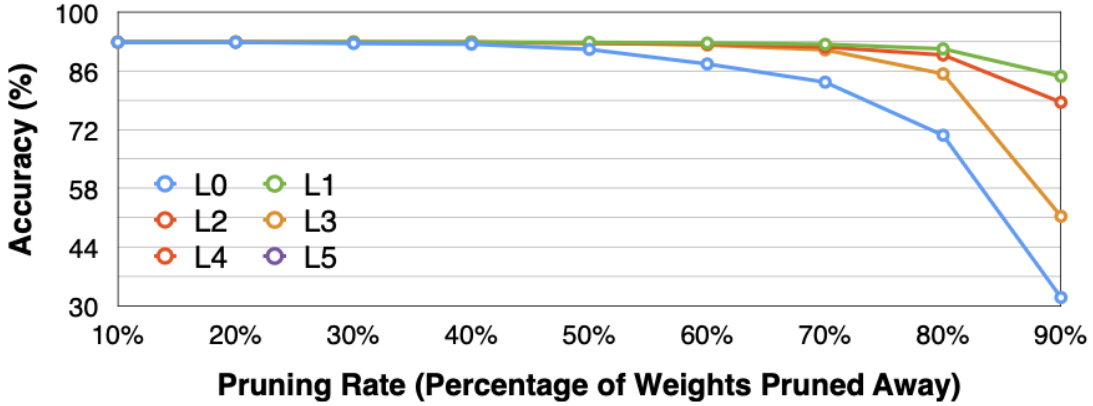
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0,0.1,0.2,...,0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers



Finding Pruning Ratios

Analyze the sensitivity of each layer

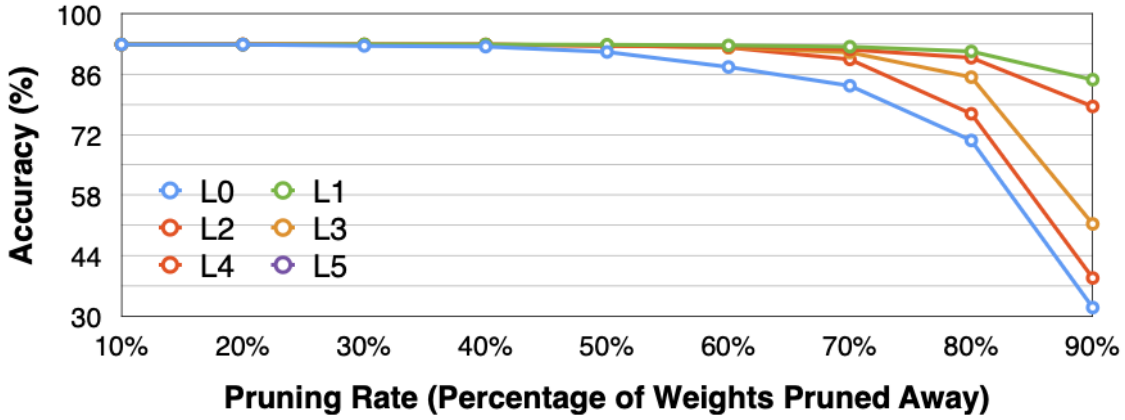
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers



Finding Pruning Ratios

Analyze the sensitivity of each layer

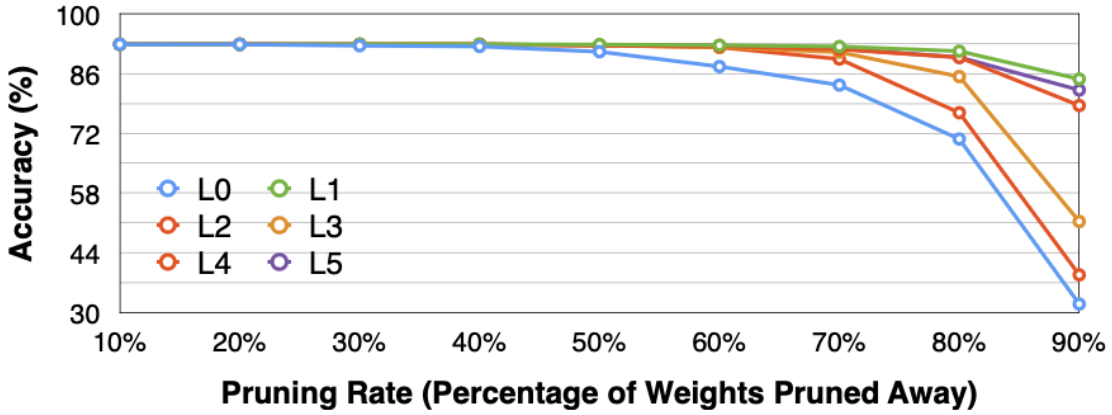
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0,0.1,0.2,...,0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers



Finding Pruning Ratios

Analyze the sensitivity of each layer

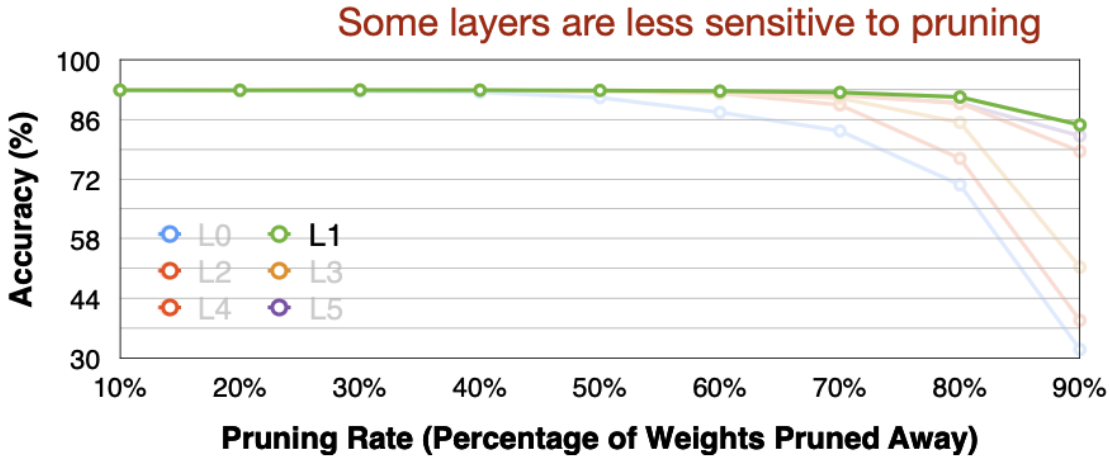
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0,0.1,0.2,...,0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers



Finding Pruning Ratios

Analyze the sensitivity of each layer

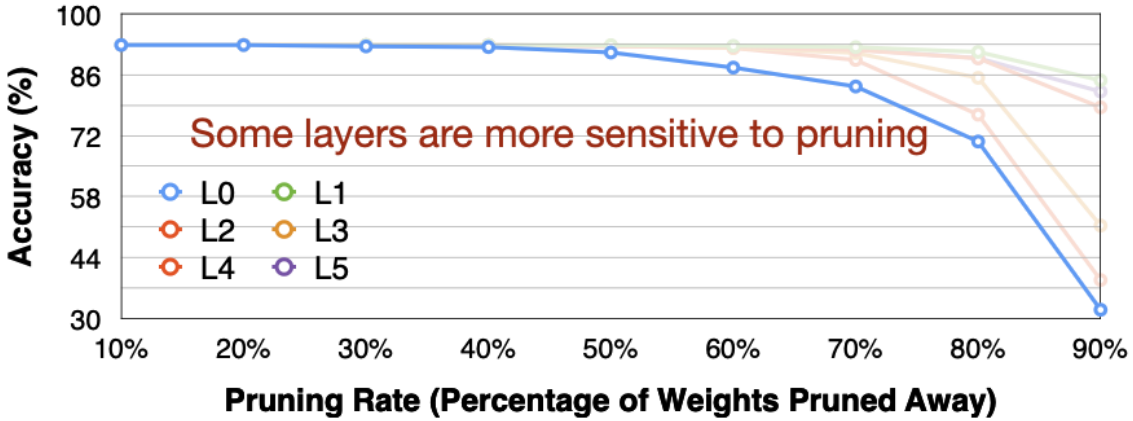
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0,0.1,0.2,...,0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers



Finding Pruning Ratios

Analyze the sensitivity of each layer

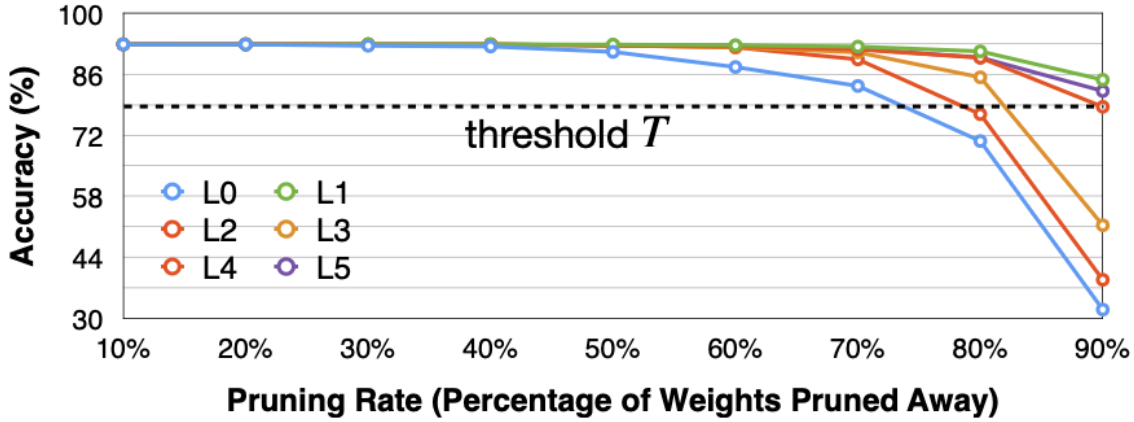
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers



Finding Pruning Ratios

Analyze the sensitivity of each layer

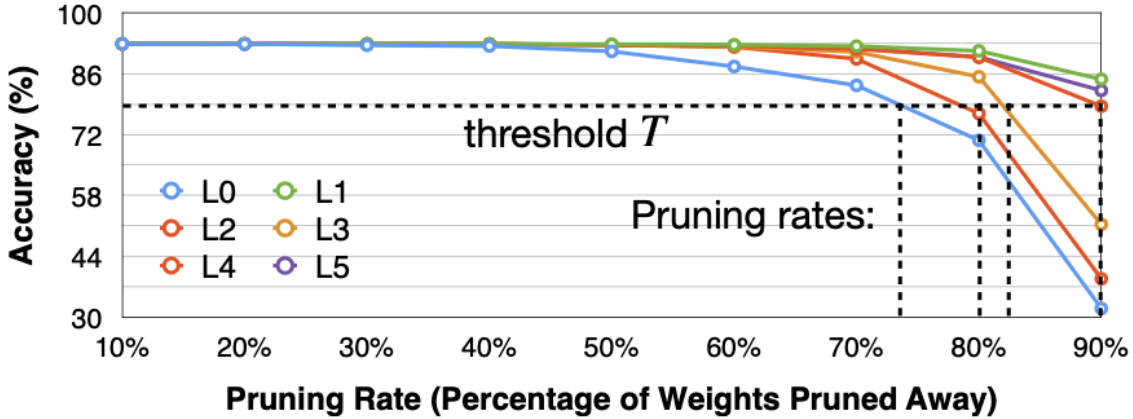
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers
  - Pick a degradation threshold T such that the overall pruning rate is desired



Finding Pruning Ratios

Analyze the sensitivity of each layer

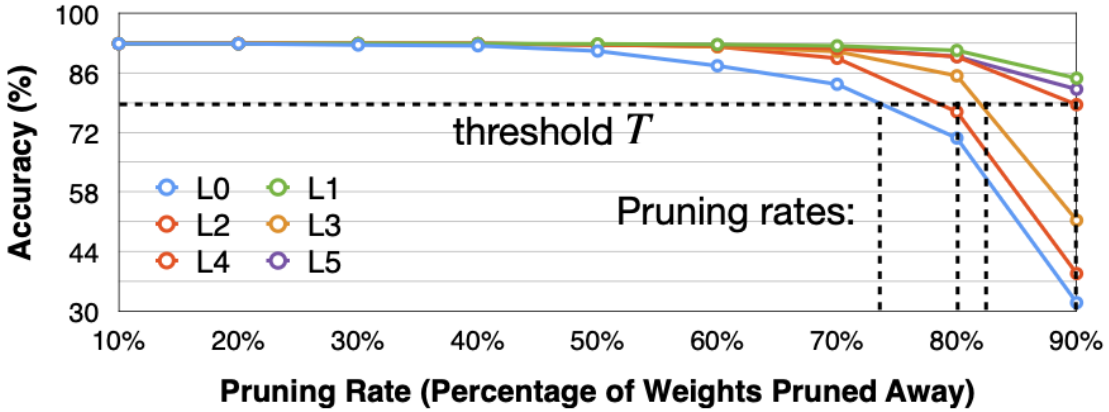
- The process of Sensitivity Analysis (* VGG-11 on CIFAR-10 dataset)
  - Pick a layer L_i in the model
    - Prune the layer L_i with pruning ratio $r \in \{0, 0.1, 0.2, ..., 0.9\}$ (or other strides)
    - Observe the accuracy degrade ΔAcc_r^i for each pruning ratio
  - Repeat the process for all layers
  - Pick a degradation threshold T such that the overall pruning rate is desired



Finding Pruning Ratios

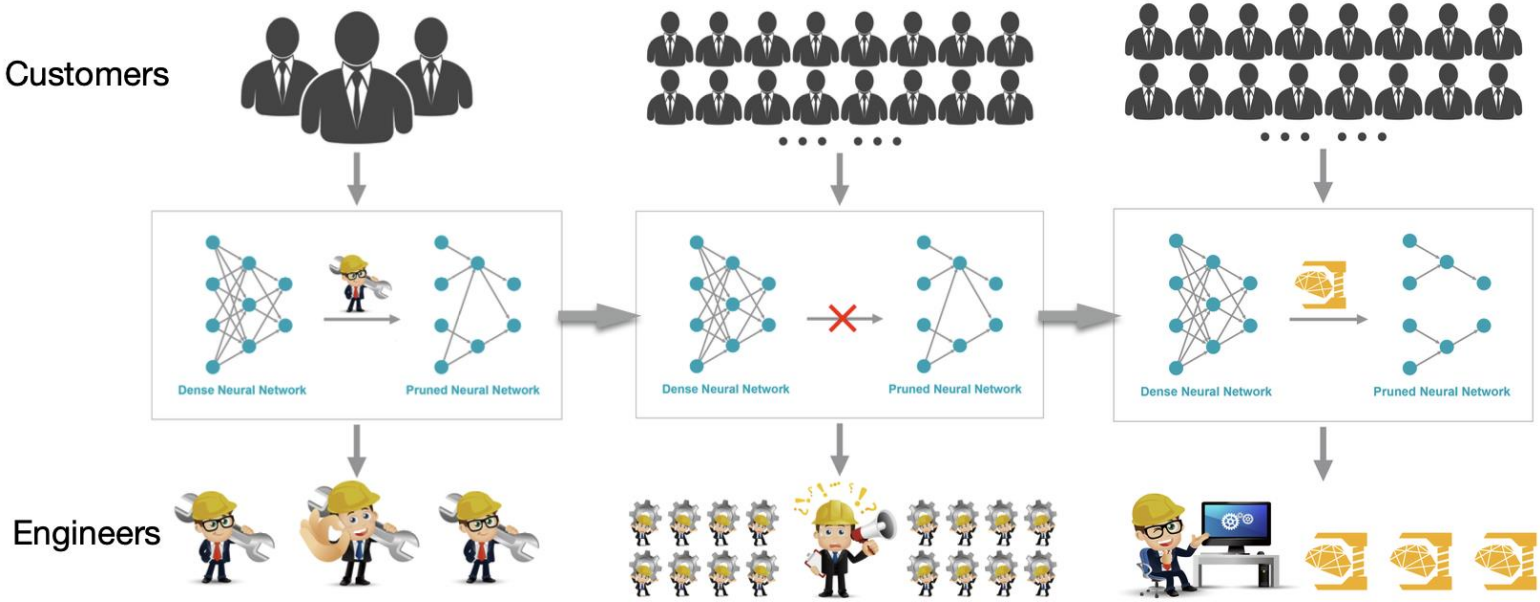
Analyze the sensitivity of each layer

- Is this optimal?
  - Maybe not. We do not consider the interaction between layers.
- Can we go beyond the heuristics?
  - Yes!



Automatic Pruning

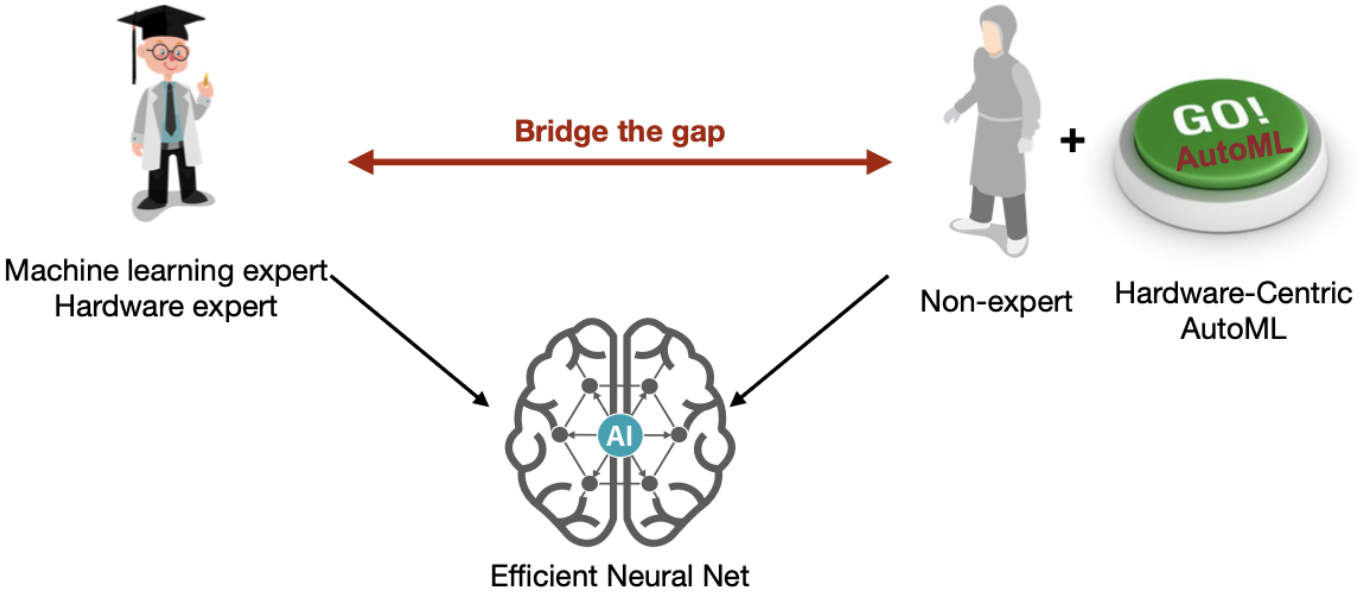
- Given an **overall** compression ratio, how do we choose **per-layer** pruning ratios?
  - Sensitivity analysis ignores the interaction between layers -> sub-optimal
- Conventionally, such process relies on human expertise and trials and errors



AMC: AutoML for Model Compression and Acceleration on Mobile Devices [He *et al.*, ECCV 2018]

Automatic Pruning

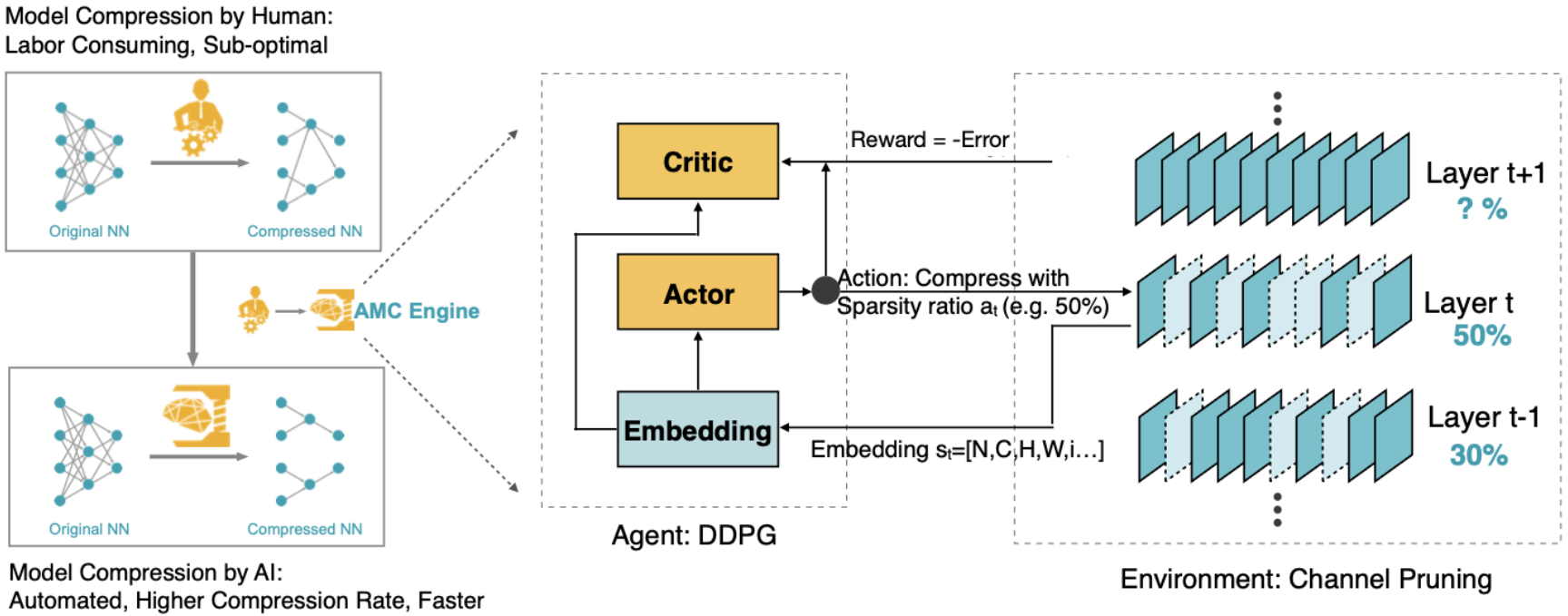
- Can we develop a push-the-button solution?



AMC: AutoML for Model Compression and Acceleration on Mobile Devices [He *et al.*, ECCV 2018]

AMC: AutoML for Model Compression

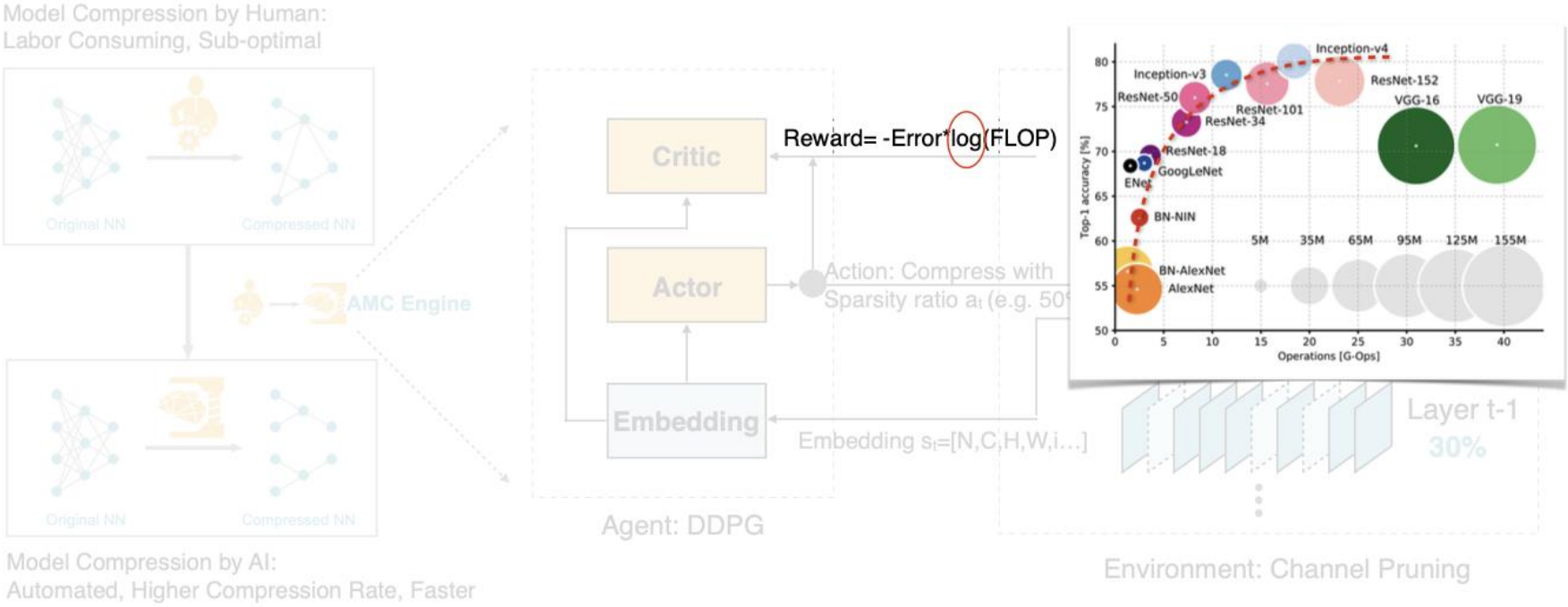
Pruning as a reinforcement learning problem



AMC: AutoML for Model Compression and Acceleration on Mobile Devices [He *et al.*, ECCV 2018]

AMC: AutoML for Model Compression

Pruning as a reinforcement learning problem



An Analysis of Deep Neural Network Models for Practical Applications [Canziani *et al.*, 2016]

AMC: AutoML for Model Compression

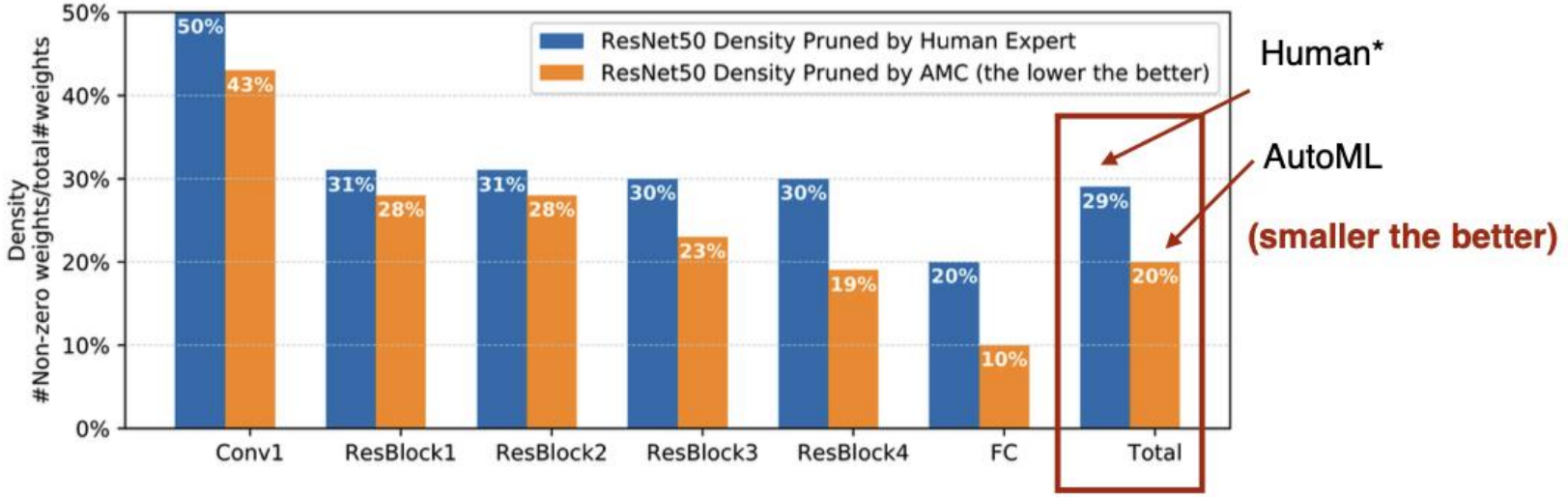
Pruning as a reinforcement learning problem

- AMC uses the following setups for the reinforcement learning problem
  - **State**: features including layer indices, channel numbers, kernel sizes, FLOPs, …
  - **Action**: A continuous number (pruning ratio) $a \in [0,1)$
  - **Agent**: DDPG agent, since it supports continuous action output
  - **Reward**: $R = \begin{cases} -\text{Error}, & \text{if satisfies constrains} \\ -\infty, & \text{if not} \end{cases}$
    - We can also optimize latency constraints with a pre-built lookup table (LUT)

AMC: AutoML for Model Compression and Acceleration on Mobile Devices [He *et al.*, ECCV 2018]

AMC: AutoML for Model Compression

Pruning as a reinforcement learning problem



* Efficient Methods and Hardware for Deep Learning [Han, *thesis*]

AMC: AutoML for Model Compression

Pruning as a reinforcement learning problem



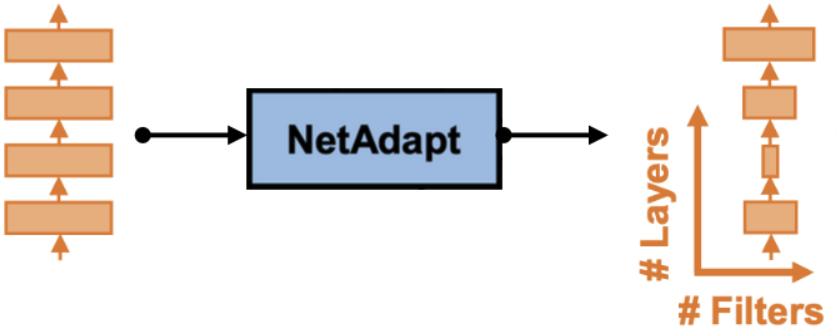
Model	MAC	Top-1	Latency*	Speedup	Memory
1.0 MobileNet	569M	70.6%	119.0ms	1x	20.1MB
AMC (50% **FLOPs**)	285M	**70.5%**	64.4ms	**1.8x**	14.3MB
AMC (50% **Time**)	272M	**70.2%**	59.7ms	**2.0x**	13.2MB
0.75 MobileNet	325M	68.4%	69.5ms	1.7x	14.8MB

* Measured with TF-Lite on Samsung Galaxy S7 Edge, which has Qualcomm Snapdragon SoC
Single core, Batch size = 1(mobile, latency oriented)

NetAdapt

A rule-based iterative/progressive method

- The goal of NetAdapt is to find a per-layer pruning ratio to meet a global resource constraint (e.g., latency, energy, …)
- The process is done iteratively
- We take **latency** constraint as an example



NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

NetAdapt

A rule-based iterative/progressive method

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)



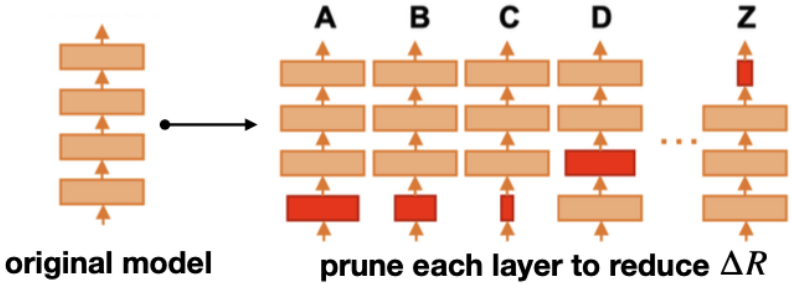
original model

NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

NetAdapt

A rule-based iterative/progressive method

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
  - For each layer L_k (k in A-Z in the figure)
    - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)

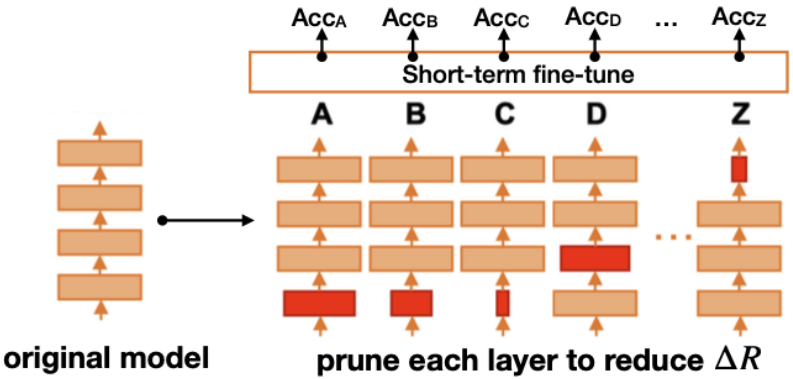


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

NetAdapt

A rule-based iterative/progressive method

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
  - For each layer L_k (k in A-Z in the figure)
    - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)
    - Short-term fine-tune model (10k iterations); measure accuracy after fine-tuning

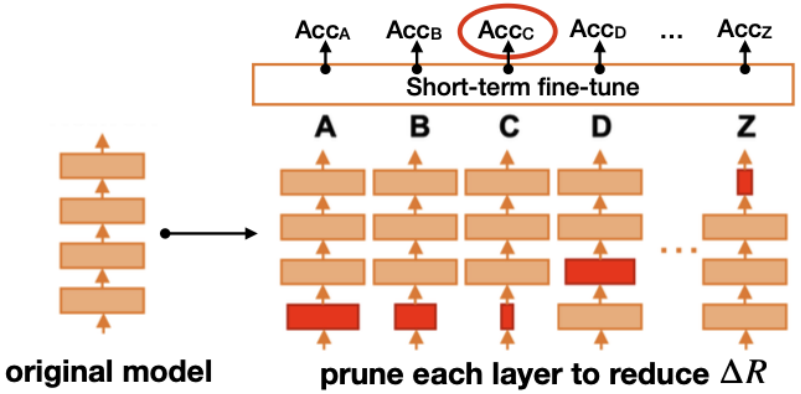


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

NetAdapt

A rule-based iterative/progressive method

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
  - For each layer L_k (k in A-Z in the figure)
    - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)
    - Short-term fine-tune model (10k iterations); measure accuracy after fine-tuning
  - Choose and prune the layer with the highest accuracy

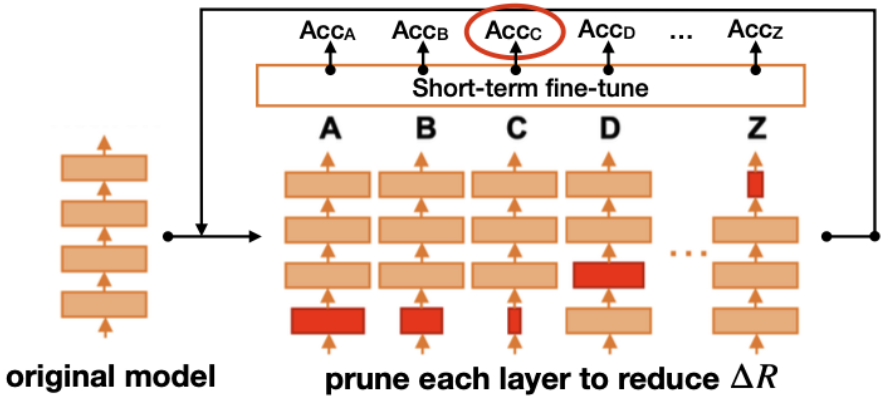


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

NetAdapt

A rule-based iterative/progressive method

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
  - For each layer L_k (k in A-Z in the figure)
    - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)
    - Short-term fine-tune model (10k iterations); measure accuracy after fine-tuning
  - Choose and prune the layer with the highest accuracy
- Repeat until the total latency reduction satisfies the constraint

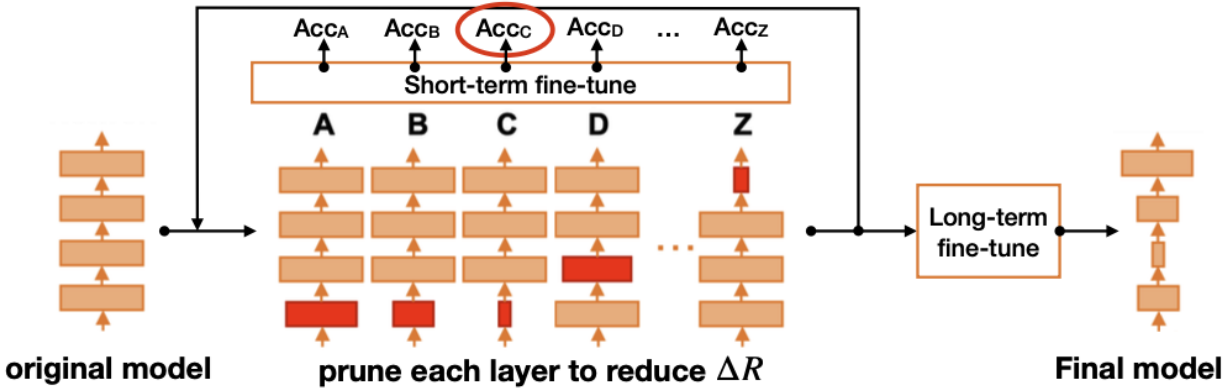


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

NetAdapt

A rule-based iterative/progressive method

- For each iteration, we aim to reduce the latency by a certain amount ΔR (manually defined)
  - For each layer L_k (k in A-Z in the figure)
    - Prune the layer s.t. the latency reduction meets ΔR (based on a pre-built lookup table)
    - Short-term fine-tune model (10k iterations); measure accuracy after fine-tuning
  - Choose and prune the layer with the highest accuracy
- Repeat until the total latency reduction satisfies the constraint
- Long-term fine-tune to recover accuracy

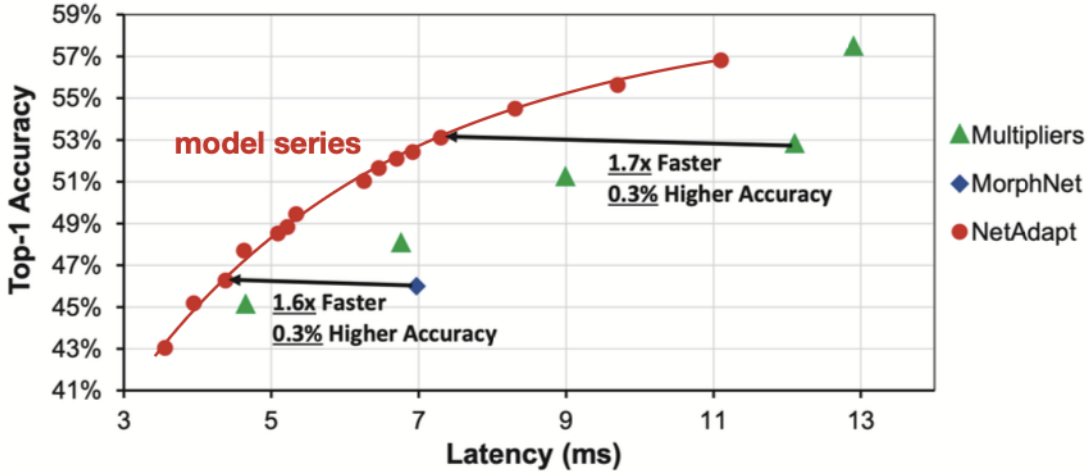


NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

NetAdapt

A rule-based iterative/progressive method

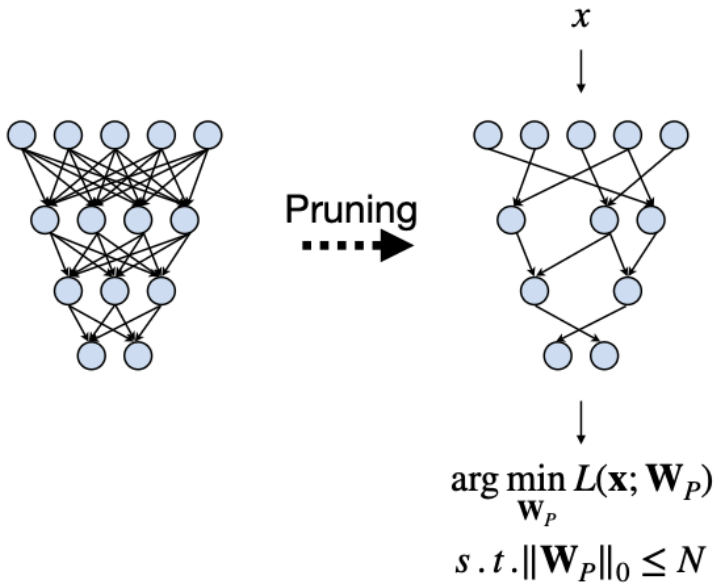
- The iterative nature allows us to obtain **a serial of** models with different costs
  - #models = #iterations



NetAdapt: Platform-Aware Neural Network Adaptation for Mobile Applications [Yang *et al.*, ECCV 2018]

Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
  - How should we improve performance of pruned models?



Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

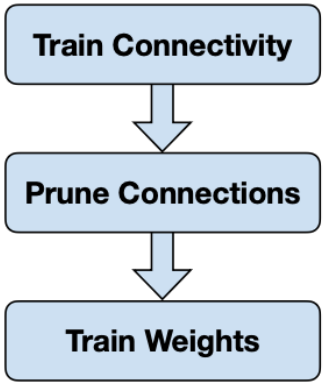
Fine-tuning / Training

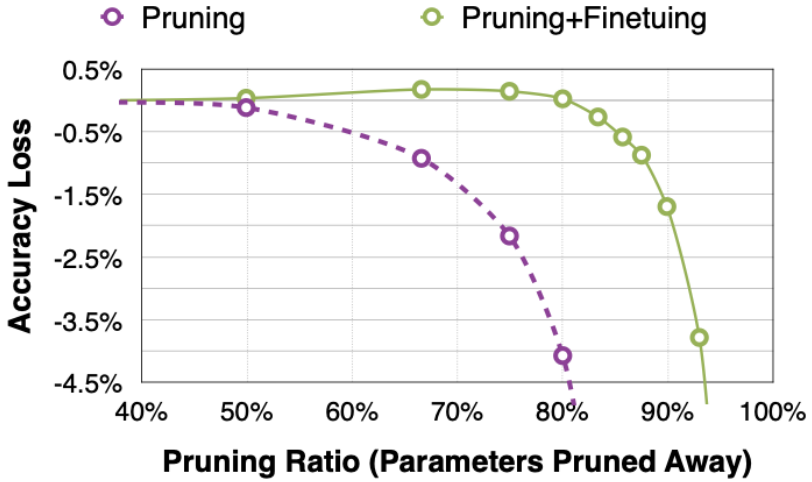
How should we improve performance of sparse models?



Finetuning Pruned Neural Networks

- After pruning, the model may decrease, especially for larger pruning ratio.
- Fine-tuning the pruned neural networks will help recover the accuracy and push the pruning ratio higher.
  - **Learning rate for fine-tuning is usually 1/100 or 1/10 of the original learning rate.**

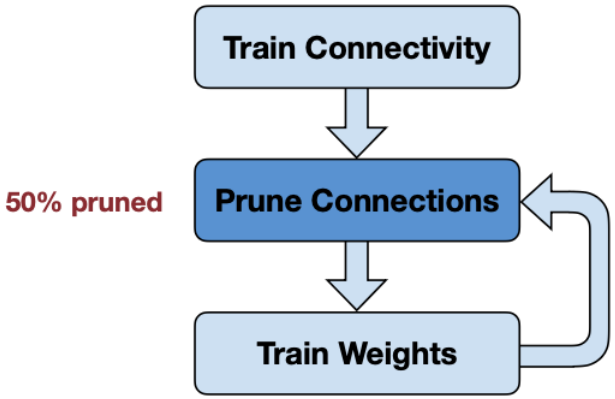


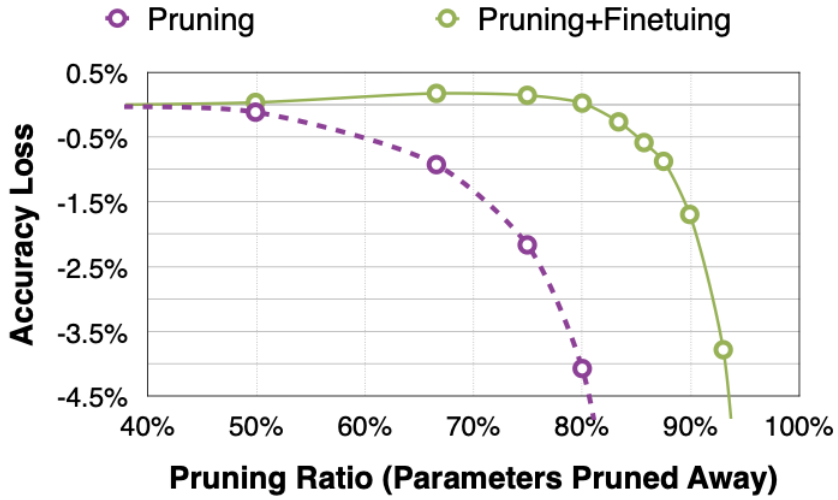


Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.

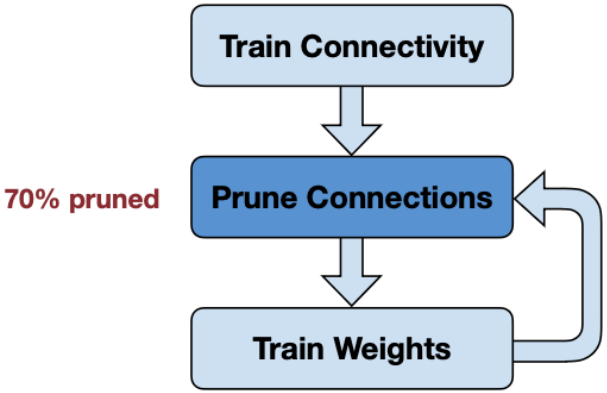


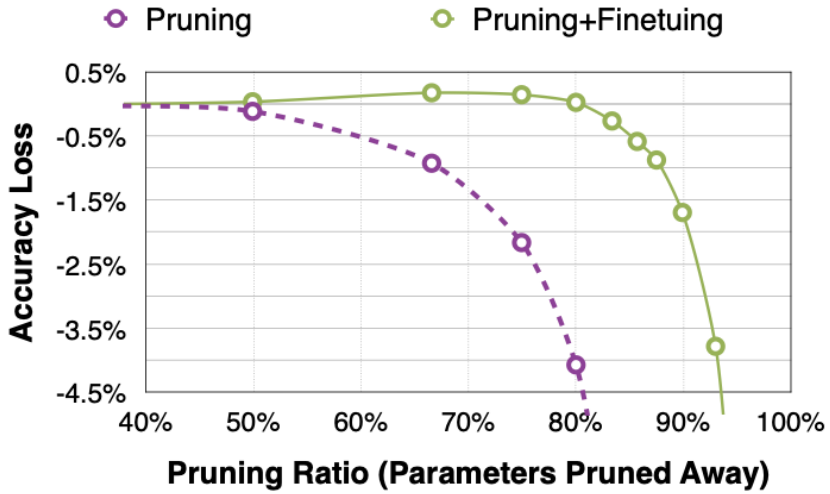


Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.

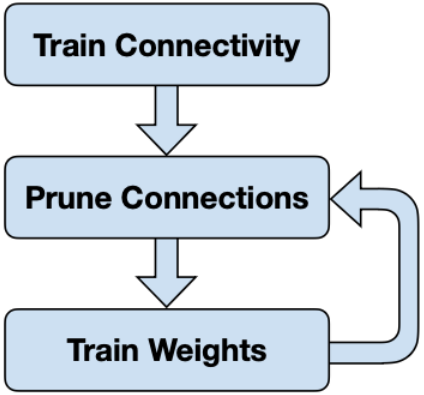


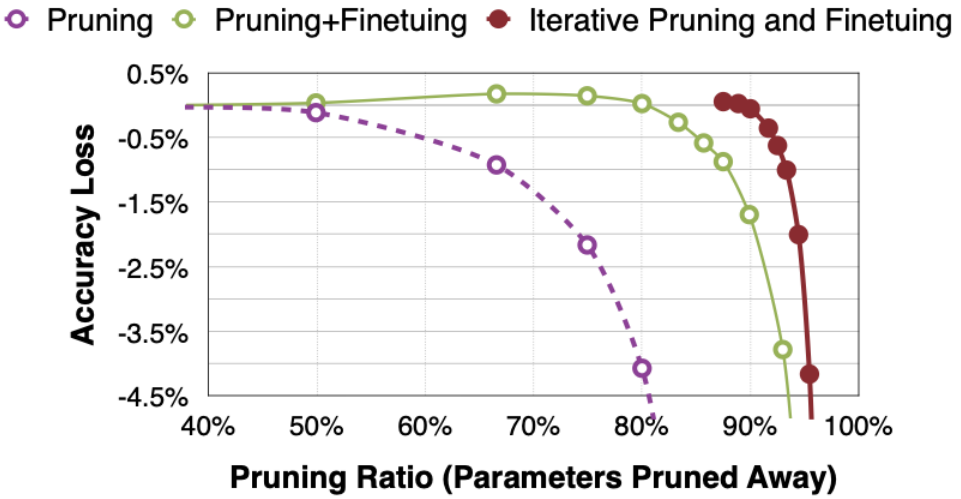


Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Iterative Pruning

- Consider pruning followed by a fine-tuning is one iteration.
- Iterative pruning gradually increases the target sparsity in each iteration.
  - boost pruning ratio from 5X to 9X on AlexNet compared to single-step aggressive pruning.

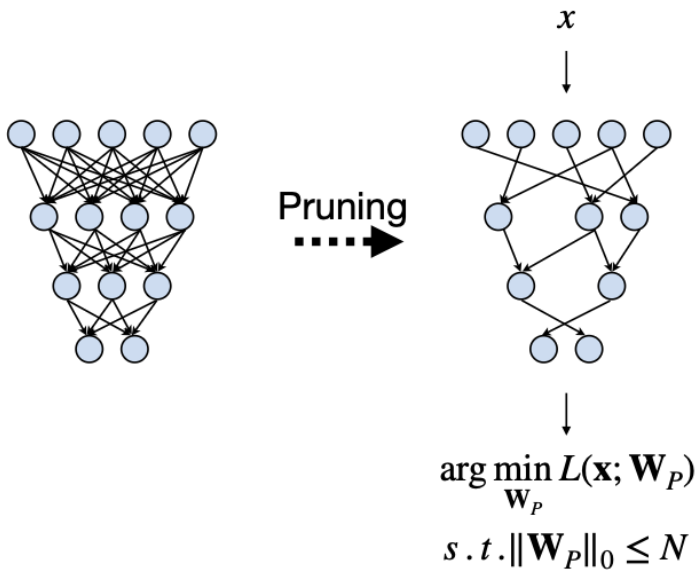




Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
  - How should we improve performance of pruned models?



Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]